A3-PS 033-TC 432

DNS-Based Discovery of Diagnostic Services

Ananya B Yakkundi
yakkundiananya@gmail.com
Akshaj Joshi
joshi.akshaj@gmail.com

AIORI 3 Hackathon

October 7,2026

Problem Description

DNS-Based Discovery of Diagnostic Services

- The Risk of "Bogus" Responses (Man-in-the-Middle Attacks): Attackers can use DNS spoofing or cache poisoning to redirect your client to a malicious server.
- The Risk of "Malformed" Responses (Data Integrity Issues): Humans make typos when entering DNS configurations, or legacy systems might update records using an incompatible schema.
- The Vulnerability of Unprotected Endpoints (Unauthorized Access): Simply discovering a valid, secure lab endpoint does not mean the client is allowed to use it.
- Decentralization and "Single Point of Failure" Bottlenecks: Traditional enterprise registries rely on a single, centralized API gateway or database to list all services.

Background

In today’s cloud-based world, servers are always moving, restarting, or scaling up and down. Every time a diagnostic server shifts, its IP address changes. If our tools have those IP addresses hardcoded into their systems, the tools instantly break. Network engineers end up playing a never-ending game of whack-a-mole just keeping lists updated.

To stop hardcoding, some companies built their own private internal registries to keep track of their servers. But the issues faced by them were: the systems were incredibly high-maintenance, they didn't talk to each other, and they completely broke down if two different networks need to test things together.

When tools try to look for endpoints automatically, they are easy targets for hackers. A bad actor could trick our tool into connecting to a fake endpoint. If this happens, our tool might accidentally hand over highly sensitive data about our network's layout and security straight to an attacker.

Literature and RFC Review

- RFC 9460 (SVCB Records):It is about a contact card that changes dynamically based on what we need. Instead of just giving us a phone number, this record tells us that: *"Here is the address, use the back door, ring the bell three times, and make sure you speak Spanish."* It tells your network client exactly what protocol, port, and security settings a service requires before it even attempts to connect, making sure no time is wasted trying the wrong connection method.
- RFC 6763 (DNS-Based Service Discovery - DNS-SD):DNS-AID adapts the underscore prefixing standard (_service._proto.domain) popularized by DNS-SD. However, it replaces the multi-step PTR → SRV → TXT lookup chain with a direct target lookup via SVCB.
- RFC 4033, RFC 4034, RFC 4035 (DNS Security Extensions - DNSSEC core) and RFC 5155 (NSEC3):DNS-AID treats (MDNSSEC as an absolute, non-negotiable requirement UST**)**. Because dynamic network diagnostic tools alter routing tables or report link latency, accepting spoofed endpoint data could lead to routing loops or Man-in-the-Middle (MitM) attacks.
- RFC 6698 (The DNS-Based Authentication of Named Entities - DANE / TLSA): To maintain zero-trust integrity as endpoints dynamically move across networks, DNS-AID leverages DANE.
- RFC 684 (Clarifications and Implementation Guidelines for DNS Security Extensions):The draft specifies strict handling of the Authenticated Data (AD) bit in the DNS header.

Proposed Solution

Core Architecture

- **Dynamic Discovery**: Replaces hardcoded IPs with standard DNSSEC-signed SRV and TXT records to track moving endpoints.
- **Three-Step Workflow**: Executes a validated DNS lookup, parses runtime application metadata, and triggers an authenticated API call.
- **Security First**: Fetches cryptographic signatures (RRSIG) alongside network routing data to verify sender authenticity.

DNS-AID Data Layout

- **Location Mapping**: SRV records dynamically direct clients to the current host node and port (e.g., node mapping for Latency and Traceroute).
- **Structured Metadata**: TXT records carry critical key-value parameters detailing the API path and required security schemas (auth=hmac-sha256).
- **Lab Services**: Provisions three distinct test capabilities: a secure Latency Probe, a Traceroute Responder, and an open Packet Capture Node.

Client Discovery Logic

- **Validation Check**: Configures the resolver with the EDNS0 DNSSEC OK flag to enforce active security checks on responses.
- **Header Enforcement**: Inspects the Authenticated Data (AD) flag inside the DNS header to verify the response was not altered in transit.
- **Metadata Parsing**: Automatically converts the raw text response into an operational dictionary containing route paths and security rules

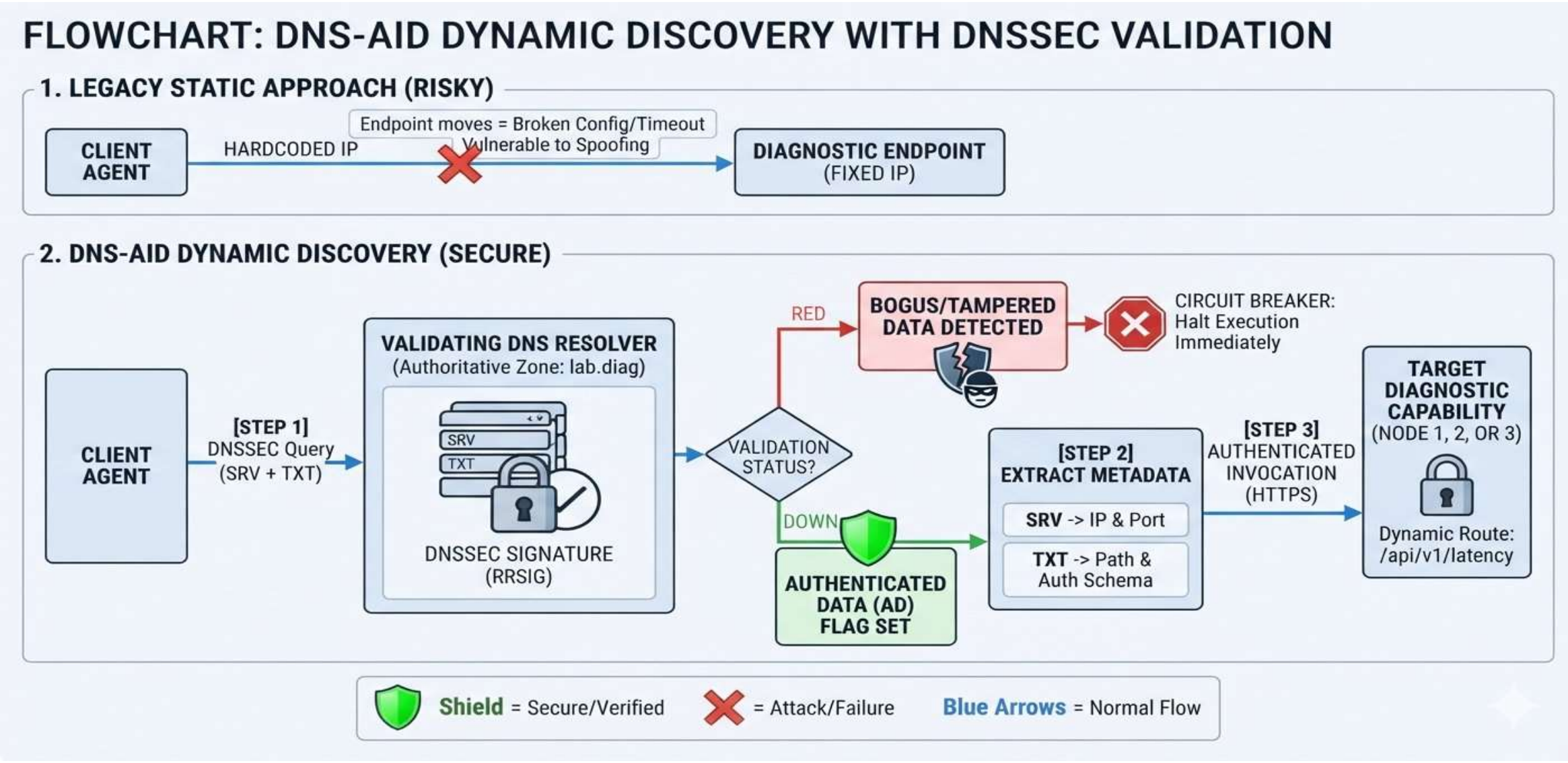
Invocation & Error Defences

- **Dynamic Authentication**: Generates time-based HMAC tokens or bearer headers on the fly, matching the exact format requested by the DNS metadata.
- **Attack Mitigation**: Halts execution instantly if validation fails, preventing the client from sending sensitive tokens to a hijacked endpoint.
- **Granular Handling**: Isolates systemic failures by distinguishing between malformed DNS-AID syntax, dead network targets, and cryptographic tampering.

Optimization Proposed

Capability / Metric	Before (The Problem)	After (The DNS-AID Solution)
Endpoint Mobility	**Hardcoded IPs** break instantly when diagnostic nodes change networks or subnets.	**Dynamic discovery** adapts automatically as nodes publish new locations to DNS
Configuration Update	**Manual updates** are required across thousands of distributed client agents.	**Zero-touch automation** updates the central zone file; clients adapt instantly.
Traffic Security	**Vulnerable to spoofing** and injection attacks due to standard unencrypted DNS lookups	**DNSSEC validation** signs records cryptographically to block data tampering.
Service Context	**IP address only**; clients lack details on target URL paths or protocols	**Rich metadata** delivers specific API paths and application parameters natively.
Authentication Hook	**Static credentials** must be pre-shared and hardwired into client codebases.	**On-the-fly execution** reads the required schema (auth=hmac) directly from metadata.
Error Resiliency	**Unpredictable failures** or data leaks occur if traffic hits a hijacked endpoint.	**Fail-safe isolation** drops the connection immediately if the security handshake fails.

Solution Architecture and Design



Data Flow Diagram

This flowchart contrasts the vulnerable legacy networking model with the secure, dynamic IETF DNS-AID architecture.

In the top section, the Legacy Static Approach demonstrates how relying on a hardcoded IP address causes configuration dropouts when an endpoint moves networks, while leaving the client open to malicious traffic spoofing.

The bottom section outlines the DNS-AID Dynamic Discovery framework, which solves this problem through a secure three-step pipeline.

First, the client issues a DNSSEC-signed query to retrieve application records backed by cryptographic signatures.

The system then evaluates the data status: if the response is tampered with or Bogus, a circuit breaker instantly drops the process to prevent a breach; if it is verified as Secure, the Authenticated Data (AD) flag is set.

Finally, the client safely extracts the live routing data and endpoint metadata from the records to perform an authenticated HTTPS invocation directly to the active diagnostic target.

Deployment Timeline

Phase 1: Environment Provisioning

- **Configure Test Zone:** Deploy a BIND9 DNS server hosting the lab.diag template zone.
- **Instantiate Mock Services:** Launch three isolated diagnostic nodes—Latency Probe, Traceroute Responder, and Packet Capture.

Phase 2: DNSSEC Hardening

- **Generate Trust Anchors:** Create Zone Signing Keys (ZSK) and Key Signing Keys (KSK) for the domain.
- **Cryptographic Zone Signing:** Execute dnssec-signzone to append RRSIG, DNSKEY, and NSEC3 records to the live zone.

Phase 3: Client Agent Engineering

- **Build Discovery Module:** Code a Python script using dnspython with the EDNS0 DNSSEC OK (DO) flag enabled.
- **Program Circuit Breaker:** Implement strict logic validating the Authenticated Data (AD) header flag to trap bogus payloads.

Phase 4: Validation & Boundary Testing

- **Validate Endpoint Mobility:** Migrate diagnostic subnets to verify the client automatically discovers new ports and API paths.
- **Inject Spoofed Responses:** Trigger a malicious DNS spoofing exploit to confirm the client successfully halts execution on tampered records.

Bibliography

- [1] .J. Mozley, N. Williams, B. Sarikaya, R. Schott, and J. Damick, "DNS for AI Discovery," IETF Internet-Draft, draft-mozleywilliams-dnsop-dnsaid-02, May 2026.
- [2] .R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "Protocol Modifications for the DNS Security Extensions," IETF RFC 4035, Mar. 2005.
- [3] B. Schwartz, M. Bishop, and E. Nygren, "Service Binding and Parameter Content Forms in the DNS (SVCB and HTTPS RRs)," IETF RFC 9460, Nov. 2023.

Thank You